

O que vamos ver nesta aula...

- criar e manipular arrays;
- usar funções como `push`, `pop`, `shift`, `unshift` e `indexOf`;
- percorrer arrays com `for`, `for...of`, `forEach`;
- transformar dados com `map`;
- filtrar, reduzir e ordenar dados;
- usar o objeto `Map`;
- aplicar tudo em um exemplo prático da plataforma de estudos.

Contexto da aplicação

Vamos criar a base lógica de uma plataforma chamada **Estuda+**.

A plataforma terá:

- lista de cursos;
- lista de notas dos alunos;
- progresso dos cursos;
- filtros;
- estatísticas;
- catálogo de planos usando Map.

Criando arrays

Um array é uma estrutura usada para guardar vários valores em uma única variável.

```
let curso = ["JavaScript", 20, 9.5, true];

console.log(typeof curso);
console.log(curso);

let curso2 = new Array("HTML", 15, 8.7, true);

let cursos = [];

console.log(cursos);
```

Agora é sua vez

Crie um array chamado `aluno` com:

- nome do aluno;
- idade;
- média;
- status de matrícula.

```
let aluno = ["Maria Eduarda", 19, 8.6, true];  
console.log(aluno);
```

Acessando posições do array

```
let curso = ["JavaScript", 20, 9.5, true];

console.log(curso[0]); // JavaScript
console.log(curso[1]); // 20
console.log(curso[2]); // 9.5
console.log(curso[3]); // true
```

O primeiro item de um array está na posição 0.

Agora é sua vez

Mostre no console apenas o nome do curso e a quantidade de aulas.

```
let curso = ["React Basico", 18, 9.2, true];  
  
console.log(curso[0]);  
console.log(curso[1]);
```

push(): adicionando no final

```
let cursos = ["HTML", "CSS", "JavaScript"];
cursos.push("React");
console.log(cursos);
```

A função `push()` adiciona um novo item ao final do array.

Agora é sua vez

Adicione o curso "Banco de Dados" ao final da lista.

```
let cursos = ["HTML", "CSS", "JavaScript"];
cursos.push("Banco de Dados");
console.log(cursos);
```


pop(): removendo do final

```
let cursos = ["HTML", "CSS", "JavaScript", "React"];
cursos.pop();
console.log(cursos);
```

A função `pop()` remove o último item do array.

Agora é sua vez

Remova o último curso da lista.

```
let cursos = ["HTML", "CSS", "JavaScript", "React"];  
cursos.pop();  
console.log(cursos);
```

100

- remover elementos;
- adicionar elementos;
- substituir elementos.

100

- **indice:** posição onde a alteração começa;
- **quantidade:** quantos itens serão removidos;
- **novoItem:** item que será adicionado.

```
let aulas = ["HTML", "CSS", "JavaScript", "React"];
aulas.splice(1, 1);
console.log(aulas);
```

Resultado esperado:

```
["HTML", "JavaScript", "React"]
```

O item "CSS" foi removido da posição 1.

```
let aulas = ["HTML", "CSS", "React"];

aulas.splice(2, 0, "JavaScript");

console.log(aulas);
```

Resultado esperado:

```
["HTML", "CSS", "JavaScript", "React"]
```

O número 0 indica que nenhum item foi removido.

Exemplo: substituindo uma aula

```
let aulas = ["HTML", "CSS", "JavaScript"];
aulas.splice(1, 1, "CSS Moderno");
console.log(aulas);
```

Resultado esperado:

```
["HTML", "CSS Moderno", "JavaScript"]
```

Agora é sua vez

Na plataforma **Estuda+**, temos a seguinte lista de aulas:

```
let aulas = ["Introdução", "HTML", "CSS", "Projeto Final"];
```

Faça as alterações usando `splice()`:

- adicione "JavaScript" antes de "Projeto Final";
- substitua "CSS" por "CSS Moderno";
- remova "Introdução".

Resultado

```
let aulas = ["Introducao", "HTML", "CSS", "Projeto Final"];

aulas.splice(3, 0, "JavaScript");
aulas.splice(2, 1, "CSS Moderno");
aulas.splice(0, 1);

console.log(aulas);
```

Resultado esperado:

```
["HTML", "CSS Moderno", "JavaScript", "Projeto Final"]
```

A lista foi alterada com splice():

- "JavaScript" foi adicionado antes de "Projeto Final";
- "CSS" foi substituído por "CSS Moderno";
- "Introducao" foi removido.

shift(): removendo do início

```
let cursos = ["Logica", "HTML", "CSS", "JavaScript"];
cursos.shift();
console.log(cursos);
```

A função `shift()` remove o primeiro item do array.

Agora é sua vez

Remova o primeiro item da trilha de estudos.

```
let trilha = ["Introducao", "HTML", "CSS", "JavaScript"];  
// remova aqui  
  
console.log(trilha);
```

Resultado

```
let trilha = ["Introducao", "HTML", "CSS", "JavaScript"];  
trilha.shift();  
console.log(trilha);
```

```
let cursos = ["HTML", "CSS", "JavaScript"];
cursos.unshift("Logica de Programacao");
console.log(cursos);
```

Agora é sua vez

Adicione "Boas-vindas" no início da trilha.

```
let trilha = ["HTML", "CSS", "JavaScript"];  
// adicione aqui  
console.log(trilha);
```

Resultado

```
let trilha = ["HTML", "CSS", "JavaScript"];  
trilha.unshift("Boas-vindas");  
console.log(trilha);
```

indexOf(): procurando posições

```
let cursos = ["HTML", "CSS", "JavaScript", "React"];

console.log(cursos.indexOf("JavaScript"));
console.log(cursos.indexOf("Python"));
```

Se o item existir, retorna sua posição.

Se não existir, retorna -1.

Agora é sua vez

Verifique se o curso "CSS" existe na plataforma.

```
let cursos = ["HTML", "CSS", "JavaScript", "React"];  
// verifique aqui
```

Resultado

```
let cursos = ["HTML", "CSS", "JavaScript", "React"];
let posicao = cursos.indexOf("CSS");
console.log(posicao);
```

```
const notas = [8.7, 6.8, 9.5, 8.8, 6.4];

let soma = 0;
let maiorNota = notas[0];

for (let i = 0; i < notas.length; i++) {
  soma = soma + notas[i];

  if (notas[i] > maiorNota) {
    maiorNota = notas[i];
  }
}

let media = soma / notas.length;

console.log(media);
console.log(maiorNota);
```

Agora é sua vez

Calcule a média de progresso dos cursos.

```
let progressos = [80, 60, 100, 40, 20];  
let soma = 0;  
  
// use for para somar os valores  
  
let media = soma / progressos.length;  
  
console.log(media);
```

Resultado

```
let progressos = [80, 60, 100, 40, 20];

let soma = 0;

for (let i = 0; i < progressos.length; i++) {
  soma = soma + progressos[i];
}

let media = soma / progressos.length;

console.log(media);
```

Percorrendo arrays com for...of

```
let cursos = ["HTML", "CSS", "JavaScript"];

for (let curso of cursos) {
  console.log(curso);
}
```

O `for...of` percorre os valores do array diretamente.

Agora é sua vez

Mostre todas as aulas da trilha no console.

```
let aulas = ["Arrays", "Map", "Filter", "Reduce"];  
// use for...of
```

Resultado

```
let aulas = ["Arrays", "Map", "Filter", "Reduce"];

for (let aula of aulas) {
  console.log(aula);
}
```


Spread: juntando arrays

```
let trilhaBase = ["HTML", "CSS"];  
let trilhaJS = ["JavaScript", "Arrays", "Map"];  
  
let trilhaCompleta = [...trilhaBase, ...trilhaJS];  
  
console.log(trilhaCompleta);
```

O spread ... espalha os elementos de um array.

© 2006 The Authors

```
let basico = ["HTML", "CSS"];
let avancado = ["React", "Node.js"];

let trilha = [...basico, ...avancado];

console.log(trilha);
```

Spread: separando caracteres

```
let palavra = "estudo";  
let letras = [...palavra];  
console.log(letras);
```

O spread também pode separar os caracteres de uma string.

```
let plataforma = "EstudaMais";

// use spread
```

Resultado

```
let plataforma = "EstudaMais";
let letras = [...plataforma];
console.log(letras);
```

Rest parameters: vários argumentos

```
function somarNotas(...notas) {
  let soma = 0;

  for (let nota of notas) {
    soma = soma + nota;
  }

  return soma;
}

console.log(somarNotas(8, 9, 7));
console.log(somarNotas(10, 6, 8, 9));
```

Aqui, ...notas empacota vários argumentos em um array.

3 1 3 1 3

```
function somarProgressos(...progressos) {
  // complete
}

console.log(somarProgressos(20, 40, 60));
```


Resultado

```
function somarProgressos(...progressos) {
  let soma = 0;

  for (let progresso of progressos) {
    soma = soma + progresso;
  }

  return soma;
}

console.log(somarProgressos(20, 40, 60));
```

```
let estudantes = ["pedro", "andre", "paulo"];

function imprimir(estudante) {
  console.log(estudante);
}

estudantes.forEach(imprimir);
```

```
let cursos = ["HTML", "CSS", "JavaScript"];

cursos.forEach(function(curso, indice, array) {
  console.log(indice);
  console.log(curso);
  console.log(array);
});
```

O callback recebe:

- elemento;
- índice;
- array completo.

100

```
let cursos = ["HTML", "CSS", "JavaScript"];

cursos.forEach(function(curso, indice) {
  console.log(`${indice + 1} - ${curso}`);
});
```

Função Simples

- Uma função simples executa uma tarefa específica.
- Ela pode ser chamada várias vezes no código.
- Nesse exemplo, a função apenas exibe uma mensagem no console.

```
function mostrarMensagem() {  
    console.log("Bem-vindo a aula de Desenvolvimento WEB Front-END!");  
}  
  
mostrarMensagem();
```

Função com Parâmetros

- Parâmetros permitem enviar informações para dentro da função.
- A função pode usar esses valores para gerar uma resposta personalizada.

```
function saudacao(nome) {  
    console.log("Ola, " + nome);  
}  
  
saudacao("Ana");  
saudacao("Carlos");
```

Função que Retorna Valor

- Uma função pode devolver um resultado usando `return`.
- O valor retornado pode ser armazenado em uma variável.

```
function somar(a, b) {  
    return a + b;  
}  
  
let resultado = somar(5, 3);  
  
console.log(resultado);
```


Função Anônima

Uma função anônima é uma função que não possui nome. Geralmente, ela é armazenada em uma variável ou passada como argumento para outra função.

```
const multiplicar = function(a, b) {  
    return a * b;  
};  
  
let resultado = multiplicar(4, 5);  
  
console.log(resultado);
```

Resultado no console:

20

Esse tipo de função é muito usado em callbacks, eventos e métodos de arrays.

Arrow Functions

Arrow function é uma forma mais curta e moderna de escrever funções em JavaScript.

Função tradicional:

```
function dobrar(numero) {  
    return numero * 2;  
}  
  
console.log(dobrar(6));
```

Arrow function equivalente:

```
const dobrar = (numero) => {  
    return numero * 2;  
};  
  
console.log(dobrar(6));
```

Quando a função possui apenas uma linha de retorno, pode ser simplificada:

```
const dobrar = numero => numero * 2;  
  
console.log(dobrar(6)); //12
```

map(): criando um novo array

```
let numeros = [1, 2, 3, 4, 5];

function quadrado(numero) {
  return Math.pow(numero, 2);
}

let quadrados = numeros.map(quadrado);

console.log(quadrados);
```

O `map()` percorre o array e cria um novo array transformado.

map(): com arrow function

```
let progressos = [10, 40, 70, 100];

let mensagens = progressos.map(progresso => {
  return `${progresso}% concluido`;
});

console.log(mensagens);
```

map(): elemento, índice e array

```
let aulas = ["Arrays", "Map", "Filter"];

let aulasNumeradas = aulas.map(function(aula, indice, array) {
  return `${indice + 1}/${array.length} - ${aula}`;
});

console.log(aulasNumeradas);
```

map(): extrair dados

```
let cursos = [
  { titulo: "HTML", aulas: 10 },
  { titulo: "CSS", aulas: 12 },
  { titulo: "JavaScript", aulas: 20 }
];

let titulos = cursos.map(curso => curso.titulo);

console.log(titulos);
```

map(): adicionar informações

```
let cursos = [
  { titulo: "HTML", progresso: 100 },
  { titulo: "CSS", progresso: 60 },
  { titulo: "JavaScript", progresso: 30 }
];

let cursosComStatus = cursos.map(curso => {
  return {
    ...curso,
    status: curso.progresso === 100 ? "Concluido" : "Em andamento"
  };
});

console.log(cursosComStatus);
```

map(): gerar HTML

```
let cursos = [
  { titulo: "HTML", aulas: 10 },
  { titulo: "CSS", aulas: 12 },
  { titulo: "JavaScript", aulas: 20 }
];

let cards = cursos.map(curso => {
  return `
    <article>
      <h2>${curso.titulo}</h2>
      <p>${curso.aulas} aulas</p>
    </article>
  `;
});

console.log(cards.join(" "));
```


Agora é sua vez

Use `map()` para criar uma lista com os títulos dos cursos em maiúsculas.

```
let cursos = [  
  { titulo: "html" },  
  { titulo: "css" },  
  { titulo: "javascript" }  
];  
  
// use map
```

Resultado

```
let cursos = [
  { titulo: "html" },
  { titulo: "css" },
  { titulo: "javascript" }
];

let titulos = cursos.map(curso => {
  return curso.titulo.toUpperCase();
});

console.log(titulos);
```

Atividade: cards da plataforma

Use `map()` para transformar cursos em cards HTML.

```
let cursos = [  
  { titulo: "HTML", categoria: "Front-end", aulas: 10 },  
  { titulo: "CSS", categoria: "Front-end", aulas: 12 },  
  { titulo: "JavaScript", categoria: "Programacao", aulas: 20 }  
];  
  
// crie os cards com map
```

Resultado

```
let cursos = [
  { titulo: "HTML", categoria: "Front-end", aulas: 10 },
  { titulo: "CSS", categoria: "Front-end", aulas: 12 },
  { titulo: "JavaScript", categoria: "Programacao", aulas: 20 }
];

let cards = cursos.map(curso => {
  return `
    <div class="card">
      <h2>${curso.titulo}</h2>
      <span>${curso.categoria}</span>
      <p>${curso.aulas} aulas</p>
    </div>
  `;
});

console.log(cards.join(" "));
```

filter(): filtrando dados

```
let progressos = [100, 80, 45, 20, 100];

let concluidos = progressos.filter(progresso => {
  return progresso === 100;
});

console.log(concluidos);
```

O `filter()` cria um novo array apenas com os itens aprovados no teste.

Agora é sua vez

Filtre somente os cursos concluídos.

```
let cursos = [  
  { titulo: "HTML", progresso: 100 },  
  { titulo: "CSS", progresso: 60 },  
  { titulo: "JavaScript", progresso: 100 }  
];  
  
// use filter
```

Resultado

```
let cursos = [
  { titulo: "HTML", progresso: 100 },
  { titulo: "CSS", progresso: 60 },
  { titulo: "JavaScript", progresso: 100 }
];

let concluidos = cursos.filter(curso => {
  return curso.progresso === 100;
});

console.log(concluidos);
```

reduce(): reduzindo para um valor

```
let valores = [1, 2, 3, 4, 5];

function soma(accumulador, valorAtual) {
  return acumulador + valorAtual;
}

let total = valores.reduce(soma);

console.log(total);
```

O `reduce()` transforma o array em um único valor.

Agora é sua vez

Calcule o total de aulas da plataforma.

```
let cursos = [
  { titulo: "HTML", aulas: 10 },
  { titulo: "CSS", aulas: 12 },
  { titulo: "JavaScript", aulas: 20 }
];

// use reduce
```

Resultado

```
let cursos = [
  { titulo: "HTML", aulas: 10 },
  { titulo: "CSS", aulas: 12 },
  { titulo: "JavaScript", aulas: 20 }
];

let totalAulas = cursos.reduce((total, curso) => {
  return total + curso.aulas;
}, 0);

console.log(totalAulas);
```

sort(): ordenando textos

```
let cursos = ["JavaScript", "HTML", "CSS", "React"];  
  
cursos.sort();  
  
console.log(cursos);
```

O `sort()` ordena os itens do array.

sort(): ordenando números

```
let notas = [8.7, 6.8, 9.5, 8.8, 6.4];  
  
notas.sort((a, b) => a - b);  
  
console.log(notas);  
  
notas.sort((a, b) => b - a);  
  
console.log(notas);
```

Por padrão, o `sort()` ordena os valores como texto, não como número, porque ele compara como se fossem strings. Logo **a - b** ordena do menor para o maior; e **b - a** ordena do maior para o menor.

Agora é sua vez

Ordene os cursos pela quantidade de aulas, do maior para o menor.

```
let cursos = [  
  { titulo: "HTML", aulas: 10 },  
  { titulo: "CSS", aulas: 12 },  
  { titulo: "JavaScript", aulas: 20 }  
];  
  
// use sort
```

Resultado

```
let cursos = [
  { titulo: "HTML", aulas: 10 },
  { titulo: "CSS", aulas: 12 },
  { titulo: "JavaScript", aulas: 20 }
];

cursos.sort((a, b) => {
  return b.aulas - a.aulas;
});

console.log(cursos);
```

Objeto Map

```
const planos = new Map([
  ["basico", 29],
  ["intermediario", 59],
  ["premium", 99]
]);

console.log(planos);
console.log(planos.size);
```

O objeto Map armazena pares de chave e valor.

Map: get(), set(), has() e delete()

```
const planos = new Map([
  ["basico", 29],
  ["intermediario", 59],
  ["premium", 99]
]);

console.log(planos.get("premium"));

planos.set("pro", 149);

console.log(planos.has("pro"));

planos.delete("pro");

console.log(planos.has("pro"));
```


Map: percorrendo dados

```
const planos = new Map([
  ["basico", 29],
  ["intermediario", 59],
  ["premium", 99]
]);

planos.forEach((valor, chave) => {
  console.log(`${chave}: R$ ${valor}`);
});
```

Agora é sua vez

Crie um Map com categorias de cursos e quantidade de cursos.

```
// exemplo:
// Front-end => 5
// Back-end => 3
// Design => 2

// crie o Map
// mostre a quantidade de cursos de Front-end
// adicione a categoria Dados
```

Resultado

```
const categorias = new Map([
  ["Front-end", 5],
  ["Back-end", 3],
  ["Design", 2]
]);

console.log(categorias.get("Front-end"));

categorias.set("Dados", 4);

categorias.forEach((quantidade, categoria) => {
  console.log(`${categoria}: ${quantidade} cursos`);
});
```

Aplicação final: base da plataforma

```
const cursos = [
  { titulo: "HTML", categoria: "Front-end", aulas: 10, progresso: 100 },
  { titulo: "CSS", categoria: "Front-end", aulas: 12, progresso: 70 },
  { titulo: "JavaScript", categoria: "Programacao", aulas: 20,
    progresso: 40 },
  { titulo: "Banco de Dados", categoria: "Back-end", aulas: 15,
    progresso: 100 }
];

const cards = cursos.map(curso => {
  let status = curso.progresso === 100 ? "Concluido" : "Em andamento";

  return `
    <article class="card">
      <h2>${curso.titulo}</h2>
      <p>${curso.categoria}</p>
      <p>${curso.aulas} aulas</p>
      <p>${curso.progresso}%</p>
      <strong>${status}</strong>
    </article>
  `;
});

console.log(cards.join(" "));
```

Aplicação final: estatísticas

```
const totalCursos = cursos.length;

const cursosConcluidos = cursos.filter(curso => {
  return curso.progreso === 100;
});

const totalAulas = cursos.reduce((total, curso) => {
  return total + curso.aulas;
}, 0);

const titulos = cursos.map(curso => {
  return curso.titulo;
});

console.log(totalCursos);
console.log(cursosConcluidos.length);
console.log(totalAulas);
console.log(titulos);
```

Resumo

Na plataforma **Estuda+**, usamos:

- arrays para armazenar listas;
- push, pop, shift e unshift para alterar listas;
- indexOf para localizar itens;
- for, for...of e forEach para percorrer dados;
- map para transformar dados;
- filter para selecionar dados;
- reduce para gerar totais;
- sort para ordenar;
- Map para armazenar pares chave-valor.

Atividade Prática: Carrinho Inteligente de Cursos

Na atividade anterior, vocês criaram uma página com cards de cursos, utilizando HTML, CSS e JavaScript.

Agora, vocês irão melhorar essa mesma página, sem precisar refazer toda a estrutura. O objetivo é adicionar uma nova funcionalidade: um **carrinho de cursos**.

Nesta atividade, vocês irão utilizar os conceitos de **array** e **map()** para armazenar, manipular e exibir os cursos adicionados ao carrinho.

O que vocês devem fazer?

Vocês deverão aproveitar os cards de cursos que já foram criados e adicionar uma área de carrinho na página.

Quando o usuário clicar no botão **Adicionar ao carrinho**, o curso escolhido deverá aparecer em uma lista de cursos selecionados.

Além disso, a página deverá mostrar:

- a quantidade de cursos adicionados;
- o nome dos cursos escolhidos;
- o preço de cada curso;
- o valor total do carrinho.

Regras da Atividade

Para desenvolver a atividade, vocês deverão seguir as regras abaixo:

- manter os cards de cursos já criados na atividade anterior;
- criar um array chamado `carrinho`;
- adicionar ao array o curso selecionado pelo usuário;
- usar `map()` para exibir os cursos adicionados ao carrinho;
- atualizar o contador de itens no carrinho;
- calcular e exibir o valor total dos cursos escolhidos.

Funcionamento Esperado

A interação da página deverá funcionar da seguinte forma:

- 1 O usuário visualiza os cards de cursos.
- 2 O usuário clica em **Adicionar ao carrinho**.
- 3 O JavaScript identifica qual curso foi selecionado.
- 4 O curso é adicionado ao array `carrinho`.
- 5 A lista do carrinho é atualizada usando `map()`.
- 6 O contador e o valor total são atualizados na tela.

Exemplo de Estrutura do Carrinho

Vocês deverão criar uma nova seção na página para exibir o carrinho.

Exemplo

Meu carrinho

Itens no carrinho: 2

- HTML e CSS Essencial - R\$ 149,90
- JavaScript para Iniciantes - R\$ 199,90

Total: R\$ 349,80

Conceitos que serão praticados

Nesta atividade, vocês irão praticar:

- criação e manipulação de arrays;
- uso de objetos para representar cursos;
- uso do método `map()`;
- eventos de clique com JavaScript;
- atualização dinâmica do HTML;
- cálculo de valores com JavaScript.

Desafio Obrigatório

Cada curso adicionado ao carrinho deverá possuir, no mínimo:

- título do curso;
- nome do instrutor;
- preço;
- identificador do curso.

Essas informações deverão ser armazenadas em objetos JavaScript dentro do array `carrinho`.

Desafios Extras

Após concluir a funcionalidade principal, vocês podem melhorar a página com uma ou mais das opções abaixo:

- impedir que o mesmo curso seja adicionado mais de uma vez;
- criar um botão para remover cursos do carrinho;
- criar um botão para limpar todo o carrinho;
- aplicar um desconto no valor total;
- mudar o texto do botão para **Adicionado**;
- destacar visualmente os cards já adicionados.

Critérios de Avaliação

A atividade será avaliada de acordo com os seguintes critérios:

- uso correto de array;
- uso correto do método `map()`;
- funcionamento do botão **Adicionar ao carrinho**;
- atualização correta da lista de cursos;
- atualização correta do contador;
- cálculo correto do valor total;
- organização e clareza do código JavaScript;
- integração com a página criada anteriormente.

Vocês deverão entregar os arquivos da página atualizada, contendo:

- arquivo HTML;
- arquivo CSS;
- arquivo JavaScript.

A página final deverá manter a estrutura visual dos cards de cursos e incluir a nova funcionalidade de carrinho interativo.

Resultado Esperado

Ao final da atividade, vocês terão uma página de cursos mais interativa, parecida com uma vitrine de uma plataforma de ensino online.

A principal diferença é que, agora, a página não apenas exibe os cursos: ela também permite que o usuário escolha cursos, acompanhe o carrinho e veja o valor total da compra.

Com isso, vocês irão perceber como arrays e `map()` ajudam a trabalhar com listas de dados em páginas web.